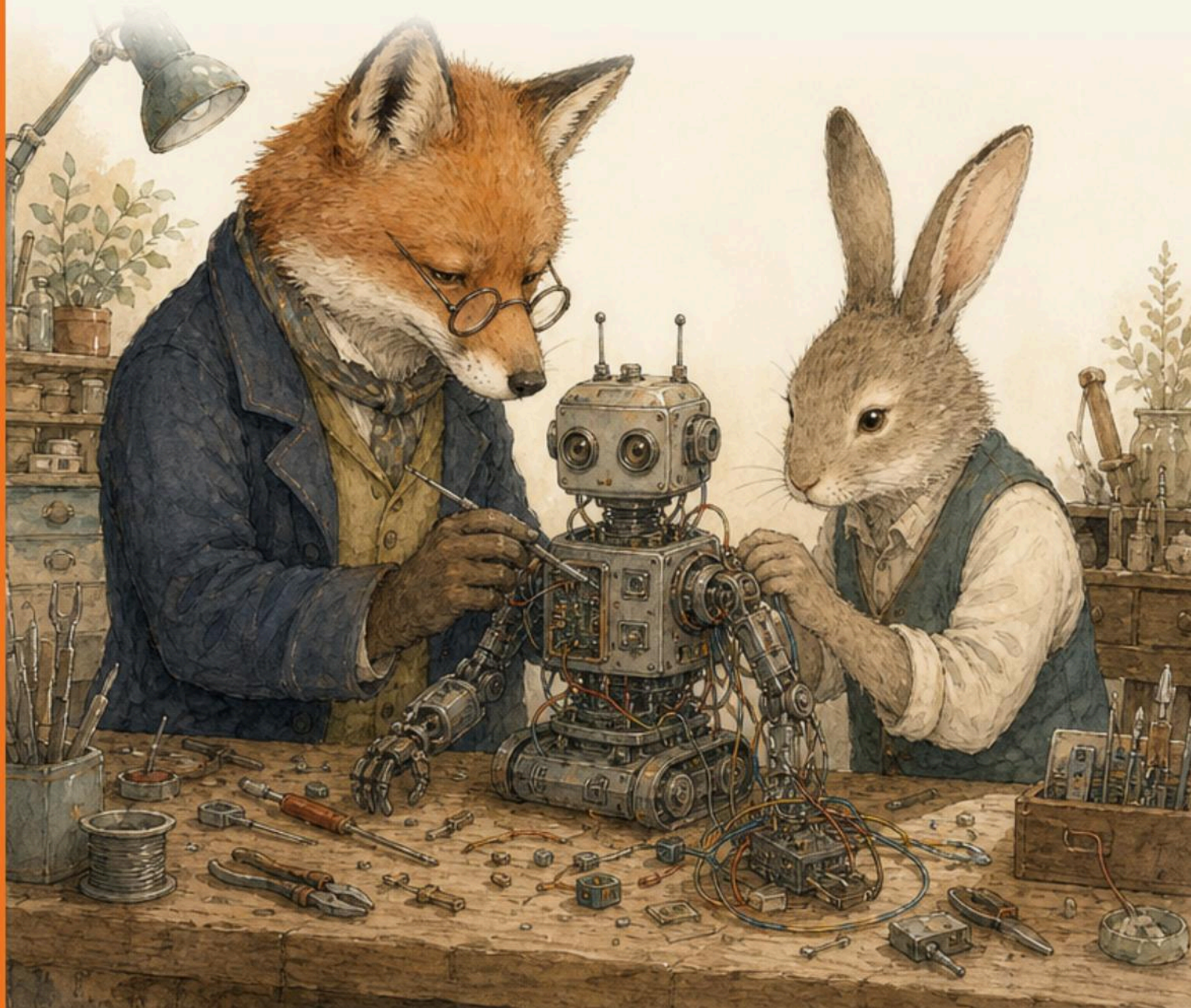
PRIME BOOKS

Computing & Robotics

Year 3

Cambridge Early Years

Student Manual



Computing & Robotics

Year 3 · Student Manual
The Mending Bench

A Cambridge Pathway student book for Lower Primary, written for pupils of about seven and eight years old.

Prime Books · Prime School International
www.primeschool.pt

First edition, 2026. Written and produced by Prime School International.

Interior 210 × 270 mm. British English throughout. Printed on uncoated stock so that pencil and crayon work on the page.

Pupils write in this book. It is meant to be written in.

Contents

Welcome to the bench

Last year you walked a path and learned to put steps in order.

This year you sit down at a **bench**.

A bench is where things get taken apart. On this bench there is a robot called Rebite who is not finished, a wall of tools that all have their own hook, and a planning board where every job is written down before anybody picks anything up.

You are the new apprentice. Your work this year is to write instructions so precise that a machine cannot get them wrong, and then to find the fault when it does anyway.

Because it will. That is not the sad part of computing. That is the craft.

> ↺ DO YOU REMEMBER?
>
> *You already know how to put steps in order, follow a simple program, and spot a pattern. Keep all of that. This year you add one word to everything you do: precise.*

Who you will meet

Mestre Raposo

The keeper of the bench. He rarely gives you the answer. Instead he asks "and then what happens?" until you find it yourself. His notebook is always open somewhere.

Leonor

The apprentice before you. Quick, curious, and happy to be wrong in public, which is why she learns faster than anybody. She tries first and checks after.

Rebite

The bench robot. Half-built at the start of this book and still being improved at the end. Does **exactly** what the instructions say, never what you meant.

You

The new apprentice. You get a pencil, a place at the bench, and permission to take things apart.

How to use this book

Every topic in this book is built the same way, so you only learn the furniture once.

On the bench today

A short story from the workshop. Read it first.

Warm up

A quick game, usually standing up, usually without a device.

Here is how it works

The teaching. Deliberately short.

Watch me work one out

Mestre Raposo does one slowly, and says why.

Check it a different way

A second route to the same answer, because one route is a trick.

Your turn

Numbered practice with real room to write.

From the notebook
A true fact off a page of Mestre Raposo's notebook.

Find the fault
Something is broken on purpose. Repair it.

One step more
For when you finish early.

Bench challenge
The hard one. Worth the effort.

No screen needed
This task needs no device at all.

Now I can
Tick these when they are true, not before.

How to work at this bench

1. Read the story, then say the key words out loud.
2. Read the teaching once, then close the book and say it back in your own words.
3. Follow the worked example with a pencil in your hand, not just your eyes.
4. Do your turn. Write in the book. It is your book.
5. When something goes wrong, do not rub it all out. Find the **one** thing that is wrong.

> ✧ FROM THE NOTEBOOK
>
> There is a rule on the wall above this bench, and it is the whole of Year 3 computing: **change one thing, then run it again.** Change three things and you will never know which one mattered.

Getting set up

◧ OPEN THE BENCH

You need very little for this book, and for the whole of Unit 1 you need no device at all.

- ✓ a pencil, and a rubber you are allowed to use
- ✓ this book, which you are allowed to write in
- ✓ a partner, because most tasks here are done out loud with somebody else
- ✓ squared paper, for when you run out of room
- ✓ a floor space or a table for grid-mat tasks

From Unit 2 onwards you will also need a tablet or a computer with Scratch, when your teacher opens it.

⬣ STOP AND THINK

When you go online later in this book, three rules never change: ask a trusted adult before you share anything, never give your full name or your school, and tell an adult straight away if something makes you uncomfortable. Unit 4 spends a whole topic on this.

§ FOR TEACHERS

This title is authored in `MARKDOWN/` and composed by the engine in `WORKSTATION/_build`. The scheme of work is `PDF/Input/Edu360 Topic (edu360.topic) Computing Y3.xlsx`, and the five units and 31 topics are kept verbatim from it. The plan the book is gated against is `MARKDOWN/06-SCHEME-MAP.md`.

> Unit 1 is entirely unplugged by design. Pupils who meet precision on paper first make far fewer guesses when Scratch arrives in Unit 2. The full curriculum mapping, the assumptions about prior learning, and the answers are in the back matter.

UNIT 1 · THE PLANNING BOARD

1

Computational thinking and programming

Think it, say it, try it, fix it.

Nobody at this bench picks up a tool before the plan is pinned to the board.

THINK IT

What must happen, and in what order?

→

SAY IT

Write the steps so precisely that nobody can guess wrong.

→

TRY IT & FIX IT

Run the steps, find the fault, change one thing.

◎ **BY THE END YOU WILL**

- ✓ say what an algorithm is, in your own words
- ✓ follow an algorithm exactly, even when you can see a shortcut
- ✓ write an algorithm precise enough for somebody else to follow
- ✓ explain what an error is, and why order matters
- ✓ find the one broken step in an algorithm and repair it

◆ ON THE BENCH TODAY

Mestre Raposo has taped a long strip of paper to the planning board. On it, in his neat handwriting, are eleven steps for building Rebite's left arm.

Leonor reads step one, picks up the wrong screwdriver, and stops. She looks again. Step one does not say *which* screwdriver.

"Then step one is not finished," says Mestre Raposo, and hands her a pencil. "Mend the step, not the arm."

Picture 1.1 Nothing is built at this bench until the plan on the board is precise.

✦ WARM UP

Stand up. Give your partner three instructions to get from their chair to the classroom door, and nothing more. Your partner must do **exactly** what you say, no guessing.

Most partners end up facing a wall. That is the lesson, not a failure.

↺ DO YOU REMEMBER?

In Year 2 you put steps in order and ran them. This year you make the steps so precise that no one has to guess what you meant.

UNIT 1 - TOPIC 1.1

Learn what an algorithm is

Here is how it works

An **algorithm** is a set of steps, in order, that gets a job done.

It is not a computer word. It is a plan word. When you clean a paintbrush, feed a cat or catch tram 28 in Lisbon, you follow steps in order. Write those steps down and you have written an algorithm.

Three things make an algorithm useful:

- it has **steps**, not a vague idea
- the steps are **in order**, and the order matters
- each step is **precise**, so two different people do the same thing

■ WATCH ME WORK ONE OUT

Mestre Raposo writes an algorithm for sharpening a pencil.

1. Hold the pencil in your writing hand.
2. Put the pencil point into the sharpener hole.
3. Turn the pencil eight times.
4. Take the pencil out and look at the point.
5. If the point is still flat, turn it four more times.

Now read step 3 again. It does not say "turn it a bit". It says **eight times**. That is what makes it precise, and precision is why somebody else can follow it and get your result.

Picture 1.2 Five precise steps, and one sharp pencil. Step 3 says how many turns, not "a few".

⇄ CHECK IT A DIFFERENT WAY

Swap the order of steps 2 and 3, then read it again: turn the pencil eight times, then put it into the sharpener. Same five steps, same words, and now the pencil is not sharp.

An algorithm is its steps **and** their order. Change one, and you have changed the plan.

✧ FROM THE NOTEBOOK

The word **algorithm** comes from a person's name. Muhammad ibn Musa al-Khwarizmi worked at the House of Wisdom in Baghdad around the year 820, and he wrote careful step-by-step methods for working with numbers. When his book was translated into Latin, his name was written *Algoritmi*, and that is the word we still use today.

He did not invent the idea of following steps in order. People had been doing that for thousands of years. His name simply stuck to it.

✂ NO SCREEN NEEDED

You have not touched a device yet in this unit, and you will not need one until Unit 2. Computational thinking happens in your head, on paper and out loud, long before it happens on a screen.

• YOUR TURN

1. Write an algorithm of four steps for washing your hands. Somebody else must be able to follow it without asking you a single question.

2. Read a classmate's hand-washing algorithm out loud and do **exactly** what it says.
Where did you have to guess? Write the step number here:

3 Which of these is an algorithm? Tick the ones that are.

a. Be tidy.

b. Put the lids on the pens, then put the pens in the pot, then close the drawer.

c. Draw a house.

d. Fold the paper in half, then in half again, then open it once.

UNIT 1 - TOPIC 1.2

Use an algorithm

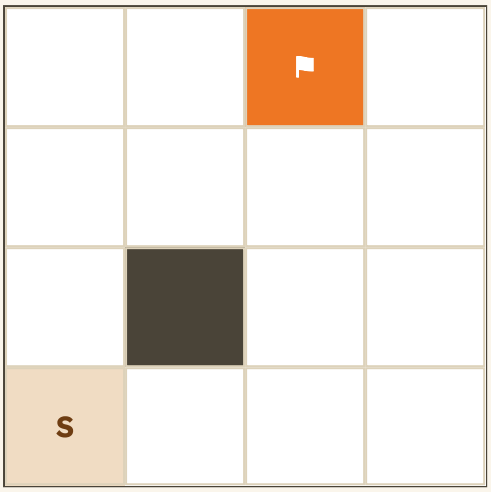
Here is how it works

Writing an algorithm is half the work. **Using** one means following it exactly, in order, without adding anything of your own.

That is harder than it sounds, because your brain wants to be helpful. A computer is never helpful. It does what the steps say.

Rebite drives on a square grid mat on the bench. Each instruction moves it one square.

Picture 1.3 Rebite waits on the start square. It will not move until you say so, and it will move exactly one square for each instruction.



■ WATCH ME WORK ONE OUT

Leonor runs the five arrows above, starting on the square marked **S** at the bottom left. She says each one out loud and moves one square only.

STEP	INSTRUCTION	WHERE REBITE IS NOW
1	up	second row from the bottom, left column
2	up	third row, left column
3	right	third row, second column
4	right	third row, third column
5	up	top row, third column, on the flag

Five instructions, five squares, and Rebite lands on the flag. Notice that Leonor never moved two squares for one arrow, even when she could see where the flag was.

⇄ CHECK IT A DIFFERENT WAY

Read the five arrows **backwards** from the flag: down, left, left, down, down. You should arrive back at **S**.

If running an algorithm forwards and backwards does not agree, one of the two runs is wrong. That is a check you can do on any route, on any mat, without asking anybody.

YOUR TURN

1. Write the arrows that take Rebite from **S** to the flag going along the bottom row first, then up the right-hand side. Use the boxes.

2. The dark square is a broken tile and Rebite may not drive on it. Write a five-arrow route that avoids it, or write *not possible* and explain why in one sentence.

3. Your partner writes six arrows in secret and reads them to you one at a time. Put your finger on **S** and follow. Where do you land? _____

→ ONE STEP MORE

Write a route for Rebite that visits the flag **and** finishes back on **S**. How many arrows does the shortest one need? Prove it by drawing the route.

UNIT 1 - TOPIC 1.3

Learn what an error is

Here is how it works

An **error** is a step that does not do what you wanted. Programmers also call an error a **bug**.

An error is not a scolding and it is not a mess. It is information. It tells you exactly where your thinking and your instructions stopped agreeing with each other.

Errors at this bench come in three kinds, and naming the kind is most of the repair:

KIND OF ERROR	WHAT IT LOOKS LIKE	EXAMPLE
A step is **missing**	the job stops early, or something is left undone	you never said "close the lid"
A step is in the **wrong order**	every step is there, and the result is still wrong	you turned the pencil before putting it in the sharpener
A step is **not precise**	two people follow it and get two different results	"move a bit"

■ WATCH ME WORK ONE OUT

Rebite must draw a square. Leonor writes:

```
1  forward 4
2  turn right
3  forward 4
```

```
4  forward 4
5  turn right
6  forward 4
```

Rebite draws two sides, then drives straight on and off the edge of the mat.

Mestre Raposo does not rewrite the whole thing. He asks one question: "which kind of error?" Step 4 should have been *turn right*, so a step is in the **wrong order**, or rather the wrong step is in that place. One word changes, and the square closes.

Naming the kind of error tells you where to look. Rewriting everything tells you nothing.

FIND THE FAULT

Here is an algorithm for making a jam sandwich. One step is in the wrong place.

1. Take two slices of bread.
2. Spread the jam on one slice.
3. Put the two slices together.
4. Open the jam jar.

Write the number of the step that is in the wrong place:

Now write the four steps in the right order.

✧ FROM THE NOTEBOOK

The first computer bug that anybody kept was a real insect. On 9 September 1947, engineers testing the Harvard Mark II computer found a moth caught in one of its switches. They taped the moth into the machine's logbook and wrote "first actual case of bug being found". That page is kept in a museum in Washington today.

Here is the joke: engineers were **already** calling a fault a bug long before 1947. That is why finding a real one was funny enough to tape down.

Picture 1.4 A bug in the logbook. The word was already in use for a fault, which is why the joke was worth taping down.

• YOUR TURN

1 Write the three kinds of error in your own words.

2. Make this step precise: "put the water in the jug".

3. Tomás says an error means he is bad at computing. Write one sentence you could say to
Tomás.

UNIT 1 - TOPIC 1.4

Find an error in algorithm

Here is how it works

Finding an error has its own algorithm, and it is the most useful one in this book.

1. **Read** the steps out loud, one at a time.
2. **Predict** what each step will do, before you run it.
3. **Run** the steps and watch for the first place the real result and your prediction
disagree.
4. **Change one thing** only.
5. **Run it again** and compare.

Step 4 is the one everybody breaks. If you change three things and the fault disappears, you do not know which change repaired it, so you have not learnt anything and it will come back.

■ WATCH ME WORK ONE OUT

Rebite must fetch a bolt from the far corner of the mat. Leonor's route sends it into the broken tile. She works through the five steps.

STEP OF THE REPAIR	WHAT LEONOR DOES	WHAT SHE FINDS
Read	says the six arrows out loud	all six are readable
Predict	says where Rebite should be after each arrow	after arrow 3 it should be one square left of the broken tile
Run	moves Rebite one square per arrow	after arrow 3 it is **on** the broken tile
Change one thing	changes arrow 3 from right to up	nothing else is touched
Run again	runs all six from the start	Rebite reaches the bolt

The fault was in arrow 3. Arrows 4, 5 and 6 were never broken, and Leonor never rewrote them.

⇄ CHECK IT A DIFFERENT WAY

Ask a classmate to run your repaired algorithm without telling them where the fault was. If they reach the same result you did, the repair holds. If they do not, your steps are still not precise enough for somebody else, which is the only test that counts.

▲ BENCH CHALLENGE

Write a six-step algorithm with **one** error hidden in it, on purpose, for a partner to find. You must be able to say which kind of error you hid.

Swap. Your partner must:

1. name the kind of error,
2. name the step number,

3 repair it by changing one thing only.

Then run the repaired algorithm together and check it does the job.

YOUR TURN

1 Here is a route for Rebite: up, up, right, right, right, up. The mat is four squares across. Predict where Rebite finishes. Then run it. Did your prediction match?

2 Write, in order, the five steps of finding an error.

3 Why should you change only one thing at a time? Answer in two sentences.

▲ SHOW THE BENCH

The morning routine, written for a robot

Rebite is going to run your classroom's morning routine. Your job is to write it precisely enough that a robot could not get it wrong.

1. **Watch and list**
   Write down every step of your real morning routine, from coming through the door to sitting down. Do not tidy it yet.

2. **Put it in order**
   Number the steps. Two steps that could happen in either order get the same number, and say why.

3. **Make it precise**
   Rewrite any step that a partner could follow in two different ways.

4. **Hide one error**
   Copy your algorithm out with one error hidden in it, and label which kind it is on a separate slip.

5. **Swap and repair**
   Trade with a partner. Find the error, name its kind, change one thing, and run the repaired routine together.

WHAT IS BEING MARKED	GETTING THERE	DONE WELL
Steps and order	most steps are there	every step is there and the order is defended
Precision	a partner had to guess once or twice	a partner followed it with no questions
Finding the error	found it with a hint	named the kind, found the step, changed one thing

BENCH WORDS

algorithm a set of steps, in order, that gets a job done

step one instruction in an algorithm

order which step comes first, second, third

precise so clear that two people do the same thing

error a step that does not do what you wanted

bug another word for an error

predict say what will happen before you run it

repair change one thing so the algorithm works

✓ NOW I CAN

- say what an algorithm is in my own words
- follow an algorithm exactly, even when I can see a shortcut
- write an algorithm a classmate can follow with no questions
- name the three kinds of error
- predict, run, and find where the two disagree
- repair an algorithm by changing one thing only

How did it go?

UNIT 1 - TOPIC	EASY	GETTING THERE	ASK AGAIN
1.1 What an algorithm is	○	○	○
1.2 Using an algorithm	○	○	○
1.3 What an error is	○	○	○
1.4 Finding an error	○	○	○

§ FOR TEACHERS

Unit 1 is deliberately unplugged from beginning to end. Pupils who meet precision on paper and out loud first make far fewer guesses when Scratch arrives in Unit 2.

Two points are worth protecting when you are short of time:

- **The wall-facing warm up.** Pupils must be allowed to fail it. The instruction that sends

a partner into a wall is the evidence for everything else in the unit.

- **Change one thing only.** Pupils will want to fix three steps at once. Insist on one, and

ask them to say what they expect before they run it again.

Vocabulary to hear in the room by the end of the unit: algorithm, step, order, precise, error, bug, predict, repair. Home task: ask pupils to write four precise steps for a job they do at home with an adult, and bring the steps in, not the job.

Word list

Every word taught in this sample, with the unit it comes from.

algorithm a set of steps, in order, that gets a job done (1.1)

bug another word for an error (1.3)

error a step that does not do what you wanted (1.3)

hardware the physical parts of a computer you can touch (front matter)

instruction one thing you tell a machine to do (1.2)

order which step comes first, second, third (1.1)

precise so clear that two people do the same thing (1.1)

predict say what will happen before you run it (1.4)

repair change one thing so the algorithm works (1.4)

run carry out the steps, one at a time, in order (1.2)

step one instruction in an algorithm (1.1)

unplugged a computing task done without any device (1.1)

§ FOR TEACHERS

The printed word list is generated from the keyword panels in the units, so it cannot drift from the body text. Adding a word to a unit adds it here automatically.

Answers

No answers are printed inside the units, so that pupils answer aloud first. They are here.

Topic 1.1

1. Open. Accept any four steps a partner can follow with no questions. Look for a number or a named object in at least one step, for example "turn the tap on" rather than "use water".
2. Open. The point is that the pupil can name a step number where they had to guess.
3. **b** and **d** are algorithms. ***a*** ("Be tidy") is an instruction with no steps. **c** ("Draw a house") is a task with no steps. Accept a pupil who argues that **c** could be an algorithm if it were broken into steps, because that is the right idea.

Topic 1.2

1. Along the bottom row first, then up: right, right, up, up, up. Five arrows, and it is the same length as the route in the worked example, which is worth pointing out.
2. **Possible.** The broken tile is in the third row, second column. Any five-arrow route that

stays clear of it, for example up, up, right, right, up as printed in the worked example, which passes through the third row third column and not the broken tile. Pupils who write "not possible" have usually assumed Rebite must travel in a straight line.

3. Open, depends on the partner's six arrows. The check is that the pupil's finger and the partner's intention agree.

Go further: the shortest route that reaches the flag and returns to the start is **ten** arrows: five out, five back. Any pupil claiming fewer has usually let Rebite move two squares on one arrow.

Topic 1.3

1. Open. Look for: a step is missing, a step is in the wrong order, a step is not precise.
2. Open. Accept anything that fixes the vagueness, for example "pour water into the jug until it reaches the line".
3. Open. Look for the idea that an error is information, not a judgement. For example: "An error tells you where your instructions and your thinking stopped agreeing. Everybody who writes instructions gets them."

Find the fault: step **4** is in the wrong place. Correct order: open the jam jar, take two slices of bread, spread the jam on one slice, put the two slices together. Accept "take two slices" before "open the jar", because both work; the fault is that the jar is opened *after* the jam is spread, which is impossible.

Topic 1.4

1. The mat is four squares across. Starting bottom left: up, up puts Rebite in the third row,

first column. Then right, right, right puts it in the fourth column. The final up puts it in the top row, fourth column. It does **not** land on the flag, which is in the third column. The third "right" is the fault.

2. Read, predict, run, change one thing, run again.

3. Open. Look for: if you change three things and it works, you do not know which change
repaired it, so you have not learnt anything and the fault can come back.

Sources and references

§ SOURCES AND REFERENCES

Every fact in this book was checked before it was printed. These are the checks.

The word algorithm (topic 1.1). Muhammad ibn Musa al-Khwarizmi, born about 780, died about 850, worked at the House of Wisdom in Baghdad around 820. The Latin translation of his book on Hindu-Arabic numerals, *Algoritmi de numero Indorum*, gave English the word algorithm from the Latin form of his name.

a. MacTutor History of Mathematics, University of St Andrews, biography of Al-Khwarizmi. https://mathshistory.st-andrews.ac.uk/Biographies/Al-Khwarizmi

b. Mehri, B., "From Al-Khwarizmi to Algorithm", *Olympiads in Informatics*, 2017, volume 11, special issue, pages 71 to 74.

The book says al-Khwarizmi's **name** gave us the word, and that he did not invent the idea of following steps in order, because step-by-step methods are thousands of years older than he is.

The first computer bug (topic 1.3). On 9 September 1947 a team testing the Harvard Mark II at Harvard University found a moth between the contacts of a relay, taped it into the machine's logbook, and

wrote "first actual case of bug being found". The page is held by the Smithsonian's National Museum of American History in Washington.

a. Computer History Museum, *This Day in History*, 9 September. https://www.computerhistory.org/tdih/september/9

b. Centre for Computing History, "First computer bug is found", 9 September 1947. https://www.computinghistory.org.uk/det/5927/First-computer-bug-is-found

The book does **not** say who found the moth. Grace Hopper was a member of the Mark II team and made the story famous in her lectures, but the museum does not attribute that logbook page to her, so naming her as the finder would teach something untrue.

The word "bug" already meant a small fault before 1947. Thomas Edison used it that way in a letter in 1878, which is why finding a real insect was funny enough to keep.

For teachers

§ ABOUT THIS BOOK

Curriculum. This book follows the Edu360 Computing Year 3 scheme of work: five units and 31 topics, kept in the scheme's own order and with the scheme's own titles. The plan the book is built and gated against is `MARKDOWN/06-SCHEME-MAP.md`, and a topic in that plan that does not reach the printed page fails the build rather than going unnoticed.

This sample. What you are holding is Unit 1 with its full front and back matter, built so that the design, the voice and the panel furniture can be judged before Units 2 to 5 are written. It is not the finished 104-page book.

Assumed prior learning. Pupils have put steps in order and followed a simple program in Year 2. They have met patterns and sequences. They are not assumed to have used Scratch, which begins in Unit 2.

Why Unit 1 has no screens at all. Pupils who meet precision on paper and out loud first guess far less when they reach a screen. Every task in Unit 1 can be done with a

pencil, a partner and a floor space. That is a deliberate choice, not a shortage of material.

Two things worth protecting when time is short.

- ✓ The wall-facing warm up. Pupils must be allowed to fail it. The instruction that sends a

partner into a wall is the evidence for everything else in the unit.

- ✓ Change one thing only. Pupils will want to repair three steps at once. Insist on one, and

ask them to say what they expect before they run it again.

Assessment. There are no high-stakes tests in this book. The traffic-light table at the end of each unit and the *Now I can* checklist are formative, for the pupil to fill in and for you to read. The project rubric marks three things only: steps and order, precision, and finding the error.

Language. British English throughout. Pupils, programme, practise as a verb and practice as a noun. The book addresses pupils directly as "you", and all adult-facing copy in it is written to teachers.

Home learning. Ask pupils to write four precise steps for a job they do at home with an adult, and to bring in the steps rather than the job.

PRIME BOOKS

Computing & Robotics

Year 3

Cambridge Early Years

Student Manual